Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Team 33: SickSense Pathfinder
Bi-Weekly Update 2

Jason Agnew, Josiah Faircloth, Samuel Ramos, Paavan Bagla

Sponsor: SICK

Faculty: Stavros Kalafatis

TA: Roman Vengas

Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

AgriPollinate

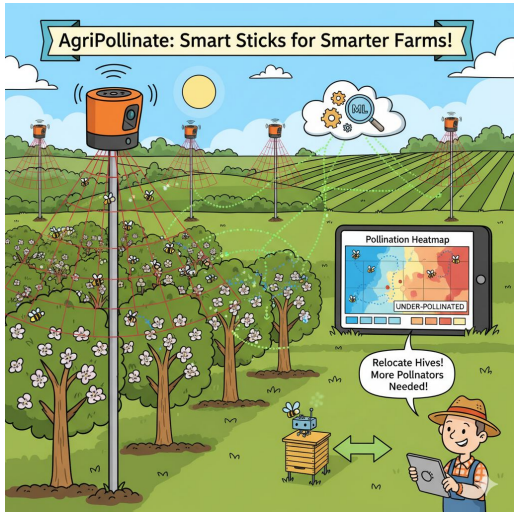
Project Summary

Problem Statement:

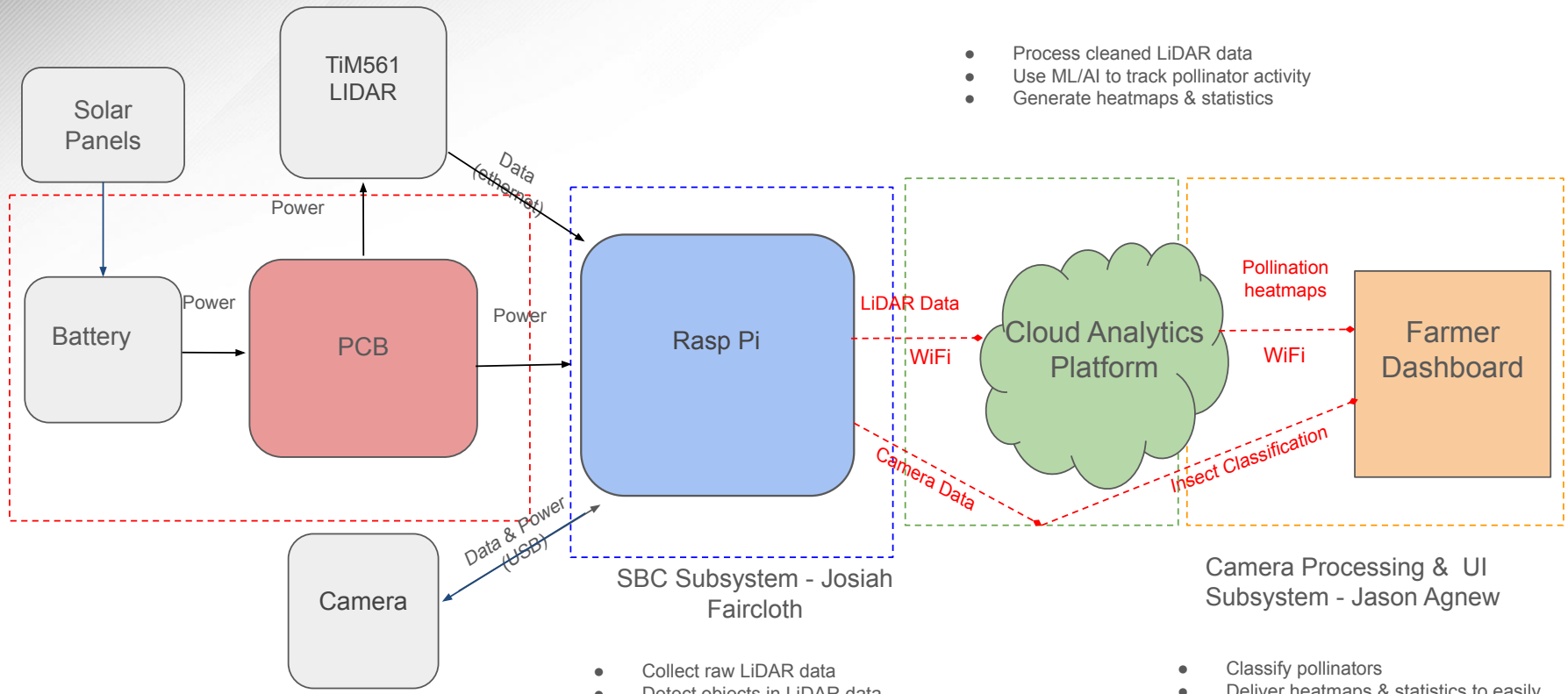
- Farmers lack reliable, real-time insights into pollination activity. Existing methods are reactive, labor-intensive, and often inaccurate, leading to missed pollination deficits that directly reduce crop yield and quality.

Our Solution:

- AgriPollinate is a Light Detection and Ranging (LiDAR)-powered monitoring system that continuously tracks pollinator activity and generates easy-to-read heatmaps.
- Farmers can access these heatmaps through a simple dashboard, allowing them to quickly identify under-pollinated areas, optimize hive placement, and improve crop yield and quality.



Subsystem Overview



LiDAR Processing Subsystem - Paavan Bagla

- Process cleaned LiDAR data
- Use ML/AI to track pollinator activity
- Generate heatmaps & statistics

SBC Subsystem - Josiah Faircloth

- Collect raw LiDAR data
- Detect objects in LiDAR data
- Capture images
- Transmit cleaned data & images

Camera Processing & UI Subsystem - Jason Agnew

- Classify pollinators
- Deliver heatmaps & statistics to easily digestible user interface

Power Subsystem - Samuel Ramos

- Supply power to physical components
- Charge Controller for the Solar Panel

Project Timeline

Subsystem reevaluation, redesigns and planning	Order PCB, basic integration preparation (completed 2/5)	Basic integration for processing systems & SBC (to complete by 2/25)	Complete Integration including PCB (to complete by 3/18)	Subsystem optimization & holistic validation (to complete by 4/1)	Final demo and Report (to complete by 4/24)

Solar Power Management Circuit

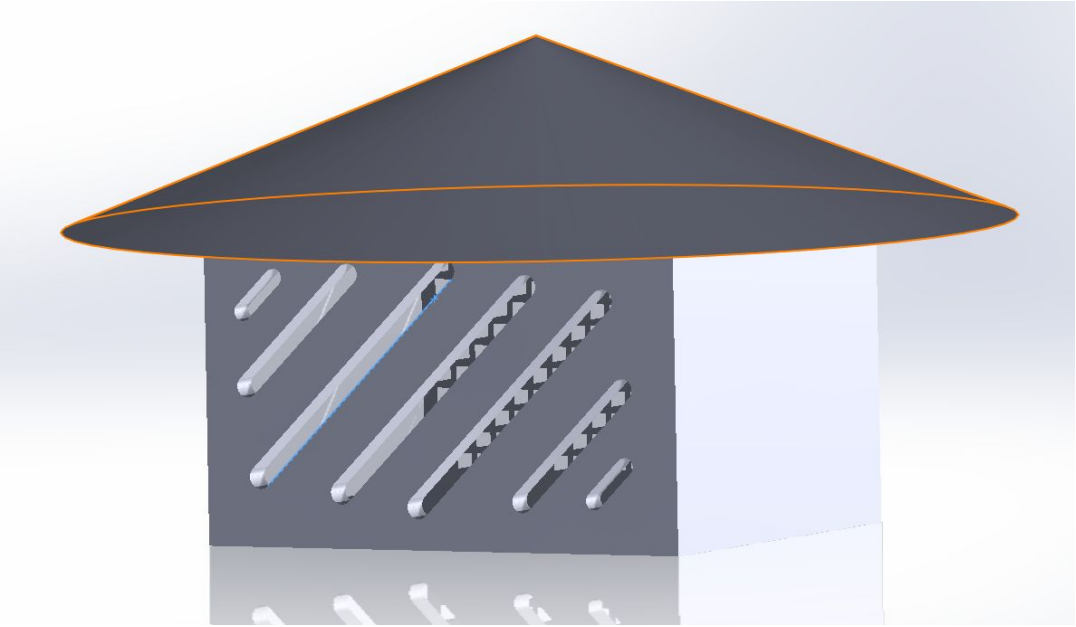
Samuel Ramos

Accomplishments since last update 15 hrs of effort	Ongoing progress/problems and plans until the next presentation
- Finalized design of the PCB - Ordering of the PCB & Components	- Soldering of said PCB - Integration of PCB into testing - Complete design of encasing for our project

Solar Power Management Circuit

Samuel Ramos

Very rough sketch!!



LiDAR Processing & Heatmaps Subsystem

Paavan Bagla

Accomplishments since last update ~14 hrs of effort	Ongoing progress/problems and plans until the next presentation
• Designed and implemented event detection logic to identify potential pollinator interactions from continuous LiDAR scans • Completed LiDAR data logging module in preparation for ML model integration	• Complete feature extraction implementation from logged LiDAR data • Generate labeled dataset for pollinator vs non pollinator events • Train initial baseline ML model • Integrate trained model into live LiDAR processing pipeline

Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

LiDAR Processing & Heatmaps Subsystem

Paavan Bagla

Capstone-Agripolinate

event_based_lidar.py | setup.py

setup.py > ...

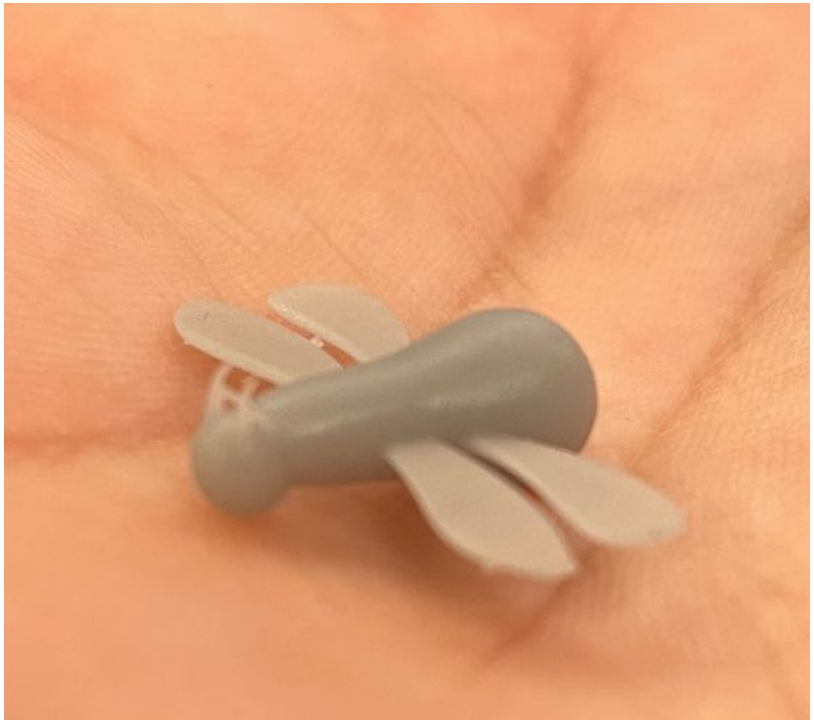
```python
import socket
import json
import time
from datetime import datetime
import statistics

HOST = "169.254.251.84"
PORT = 2112

CMD_START = b"\x02sEN LMDscandata 1\x03"
CMD_STOP = b"\x02sEN LMDscandata 0\x03"

NUM_BASELINE_SCANS = 100

DROP_THRESHOLD = 0.25

MIN_BLOCK_SIZE = 3
MAX_BLOCK_SIZE = 10

MAX_DIST_STD = 0.05


def parse_scan(telegram):
    parts = telegram.strip().split(" ")

    if "DIST1" not in parts:
        return None

    try:
        idx = parts.index("DIST1") + 5
        count = int(parts[idx], 16)

        return [
            int(v, 16) / 1000.0
            for v in parts[idx + 1 : idx + 1 + count]
        ]
    except Exception:
        return None


def find_blocks(indices):
    if not indices:
        return []

    blocks = []
```

Ln 170, Col 1 Spa

LiDAR Processing & Heatmaps Subsystem

Paavan Bagla

CAD Models: Beetle & Honeybee



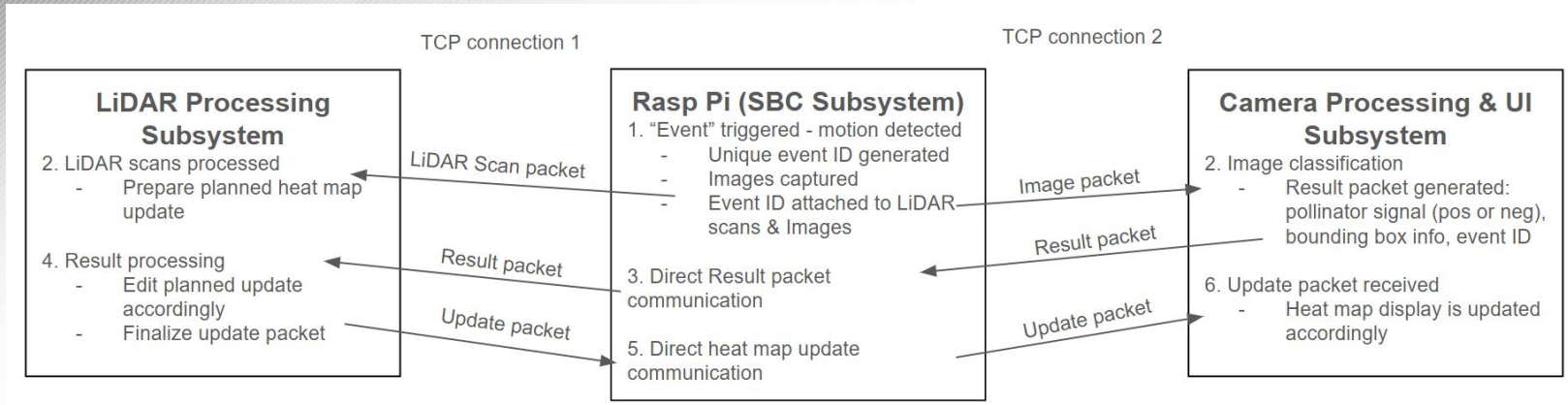
Single Board Computer Subsystem

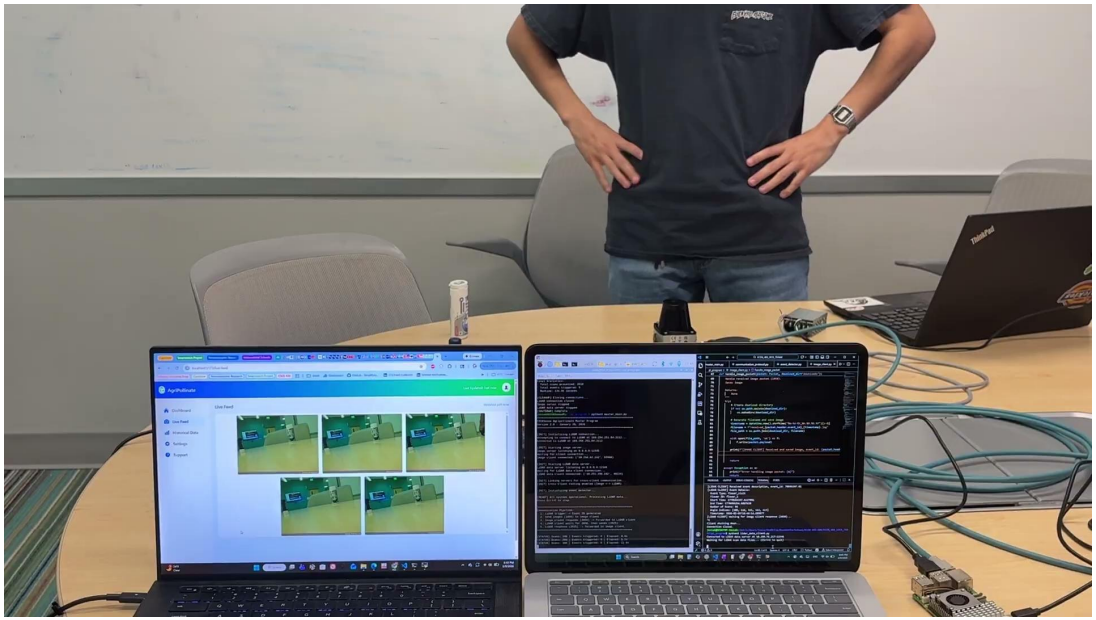
Josiah Faircloth

Accomplishments since last update ~15 hrs of effort	Ongoing progress/problems and plans until the next presentation
Debugged and integrated communication data flow - Finalized protocol programs - Finalized camera/UI communication - Modified main program to standardize LiDAR data formatting	Assist basic integration steps between camera & LiDAR processing systems Test & improve detection algorithm for accuracy and efficiency, optimize parameters, evaluate potential new filtering features to improve performance

SCB Subsystem

Josiah Faircloth





Camera Processing & UI Subsystem

Jason Agnew

Accomplishments since last update ~14 hrs of effort	Ongoing progress/problems and plans until the next presentation
• Evaluated detection model on validation data • Built initial Python API/inference script (FastAPI) for local testing • Implemented the new TCP protocol in the Node.js backend to receive multiple images from the Raspberry Pi, return classification results, and handle heatmap data from the LiDAR subsystem.	• Continue gathering additional field images, especially including other insect species, to expand training data and improve model accuracy and robustness. • Fetch images from Raspberry Pi via TCP in Node.js, POST images to FastAPI for YOLOv8 detection, send detection results back to the Raspberry Pi over TCP. • Work on formatting of heatmap in frontend using simulated heatmap

Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Camera Processing & UI Subsystem

Jason Agnew

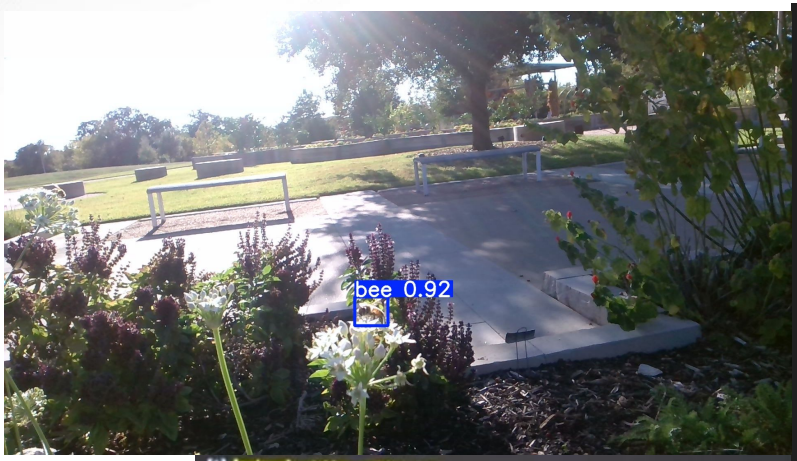
Summary metrics:

Mean precision: 0.794

Mean recall: 0.891

mAP@0.5: 0.882

mAP@0.5:0.95: 0.282




```python
from fastapi import FastAPI, File, UploadFile
from ultralytics import YOLO
from PIL import Image
import io

app = FastAPI()
model = YOLO("best.pt") # Load pre-trained YOLO model
@app.post("/detect")
async def detect(file: UploadFile = File(...)):
    img_bytes = await file.read()
    img = Image.open(io.BytesIO(img_bytes)).convert("RGB")
    results = model(img, conf=0.25)[0]

    detections = []
    for box in results.boxes:
        cls_id = int(box.cls[0])
        conf = float(box.conf[0])
        xyxy = box.xyxy[0].tolist()  # [x1, y1, x2, y2]
        detections.append({
            "class_id": cls_id,
            "confidence": conf,
            "bbox": xyxy
        })
    return {"detections": detections}
```

```javascript
// Send classification result back to Raspberry Pi server
fetch('http://localhost:3000/api/classify', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    classIdx: speciesIdx,
    roleIdx: roleIdx,
    boxCoords: {
      x1: boxCoords.left,
      y1: boxCoords.top,
      x2: boxCoords.left + boxCoords.bw,
      y2: boxCoords.top + boxCoords.bh
    }
  })
}).catch(err => {
  console.error('Failed to send classification to server:', err);
});
```

```
[1] [nodemon] watching path(s): *.*
[1] [nodemon] watching extensions: js,mjs,json
[1] [nodemon] starting `node server2.js`
[0]
[0]  VITE v7.1.10  ready in 2687 ms
[0]
[0]  ➜  Local:   http://localhost:5173/
[0]  ➜  Network: http://10.255.255.254:5173/
[0]  ➜  Network: http://172.28.7.244:5173/
[1] Waiting for image packets... (Ctrl+C to quit)
[1]
[1] Express server running at http://localhost:3000
[1] Image files served from: /mnt/c/Users/jrace/OneDrive/Desktop/ECEN_403/ECEN_403_SICK_TEAM2/agripollinate_webpage/farmer-dashboard-backend/downloads
[1] Connected to server at 10.250.76.217:12345
[0] [baseline-browser-mapping] The data in this module is over two months old.  To ensure accurate Baseline data, please update: `npm i baseline-browser-mapping@latest -D`
[0] /*! 🌼 daisyUI 5.5.17 */
[1] [DEBUG] Deserializing complete packet of 118969 bytes
[1] [IMAGE CLIENT] Received and saved image: received_70684335.02_02-09-2026_18.45.36.733.jpg, event_id: 70684335.02
[1] [DEBUG] Deserializing complete packet of 119047 bytes
[1] [IMAGE CLIENT] Received and saved image: received_70684335.02_02-09-2026_18.45.36.762.jpg, event_id: 70684335.02
[1] [DEBUG] Deserializing complete packet of 119275 bytes
[1] [IMAGE CLIENT] Received and saved image: received_70684335.02_02-09-2026_18.45.36.768.jpg, event_id: 70684335.02
[1] [DEBUG] Deserializing complete packet of 118880 bytes
[1] [IMAGE CLIENT] Received and saved image: received_70684335.02_02-09-2026_18.45.36.787.jpg, event_id: 70684335.02
[1] [DEBUG] Deserializing complete packet of 118896 bytes
[1] [IMAGE CLIENT] Received and saved image: received_70684335.02_02-09-2026_18.45.36.803.jpg, event_id: 70684335.02
[1] [BACKEND] Sent response (2050), event_id: 70684335.02
[1]
[1] [DEBUG] Deserializing complete packet of 147 bytes
[1] [LIDAR RESPONSE] Event 70684335.02: {"status":"processed","event_id":"70684335.02","timestamp":"2026-02-09T18:45:45.053059"}
```

Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Execution Plan

Assigned	Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	Week 13	Week 14
		1/14/2026	1/21/2026	1/28/2026	2/4/2026	2/11/2026	2/18/2026	2/25/2026	3/4/2026	3/18/2026	3/25/2026	4/1/2026	4/8/2026	4/15/2026	4/22/2026
Samuel	Redesigned PCB with minimal swap in components														
Josiah	Communication Management Between Subsystems														
Paavan	Define LiDAR ML sample structure and SBC communication requirements for pollination event classification														
Jason	Set up YOLOv8 training/testing environment on HPRC (conda, pip, etc.)														
Jason	Train first YOLOv8 "bees only" detection model														
TEAM	**Status Update 1**														
Samuel	Simulation of new schematics and final edits														
Paavan	Implement flower occupancy event extraction and verify event duration and stability metrics														
Paavan	Complete reliable LiDAR event communication from Raspberry Pi to SBC as specified in subsystem interface definition														
Jason	Evaluate model on validation images														
Jason	Build initial Python API/inference script (FastAPI) for local testing														
Jason	Develop the TCP protocol implementation (send detection to Raspberry Pi)														
TEAM	**Status Update 2**														
Samuel	Soldering of new PCB														
Josiah	Algorithm Improvement & Data Reduction														
Paavan	Integrate trained ML classifier with live LiDAR event stream and validate end to end event classification output														
Jason	Work on formatting of heatmap in frontend using simulated heatmap														
TEAM	Basic Integration														
TEAM	**Capstone Design Blitz**														
Samuel	Testing of new PCB under various conditions														
TEAM	**Status Update 3**														
Samuel	Finalize the design for our project using CAD software														
Samuel	3D printing of the final design														
Josiah	Memory Management & Data Retention														
Paavan	Validate full LiDAR → event detection → ML classification → structured output pipeline under controlled test scenarios														
Jason	Fetch images from Raspberry Pi via TCP in Node.js, POST images to FastAPI for YOLOv8 detection, send detection results back to the Raspberry Pi over TCP														
TEAM	**Status Update 4**														
Samuel	Debugging PCB														
Josiah	Automatic Network Handling														
Paavan	Integrate real time pollination event outputs with spatial mapping module to generate live heatmap data structure														
Jason	Continue collecting/annotating field data (including possible backgrounds)														
TEAM	**Status Update 5**														
Samuel	Final validation testing														
Josiah	Edge Case & Fault Tolerance Programming														
Paavan	Demonstrate & Validate fully integrated LiDAR based pollination heatmap during live system test														
Jason	Refine YOLO model (tune hyperparameters, do new training runs as annotations grow)														
TEAM	**Final Presentation**														
TEAM	**Project Showcase (Final Demo)**														
TEAM	**Final Report**														

Not Started
Completed
In Progress

Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

Validation Plan

Validation Plan

Paragraph #	Test Name	Success Criteria	Methodology	Status	Responsible Engineer(s)
3.2.3.3	LiDAR Scan Acquisition and Stability	Continuous LiDAR scans received and logged with no dropped frames for ≥ 10 minutes	Run LiDAR on Raspberry Pi and log scan timestamps to verify stable scan rate	Completed	Paavan
3.2.1.5	Pollination Event Extraction Validation	Persistent occupancy events detected at known flower locations for ≥ N consecutive scans	Place small object at calibrated flower positions and verify event start, duration, and end logic	Completed	Paavan
3.2.1.10	LiDAR ML Sample Integrity Test	Generated ML samples contain correct angle ranges, distance series, and event durations	Log extracted events and manually inspect sample structure and timing consistency	Work in Progress	Paavan
3.2.1.5	Pollinator Classification Validation	ML model correctly classifies pollinator vs non pollinator events with ≥ target accuracy and ≤ target false positive rate	Replay labeled LiDAR event dataset through full pipeline and compare predicted labels to ground truth annotations	Planned	Paavan
3.2.1	End to End System Integration Test	Continuous operation of full pipeline for ≥ 15 minutes with correct event detection, classification, and spatial logging and no system crashes	Run live LiDAR → event detection → ML classification → heatmap generation pipeline and monitor logs, latency, and output consistency	Planned	Paavan
3.2.1.18	Power Efficiency	The system shall operate for a minimum of 6 hours on a single battery chargeor through solar-assisted power.	Continusouly have a load on the PCB which will run for 6 hours	Planned	Samuel
3.2.1.19	Power Delivery	The system shall deliver the correct amount of power to the LiDAR and the Raspberry Pi continuously for 6 hours	While running the load, consisitently check the voltage and current rates at timed intervals	Planned	Samuel
3.2.1.20	Charging Delivery	The solar panel should be able to charge the battery used in the system with at most 2.1 amps	Measuer shunt resistor while battery is in 3 states, depleted, no load, and with load	Planned	Samuel
3.2.1.16	Communication latency	At least 99% of messages are delivered correctly with end-to-end latency under 100 ms and no ordering errors.	Perform sustained bidirectional message exchanges under normal and high-load conditions while monitoring delivery rates, latency, and message order.	Completed	Josiah
3.2.1.4	Detection performance	False positives account for no more than 20–30% of detection triggers while bee detection recall remains at or above 90%.	Replay labeled LiDAR datasets through the updated algorithm and compare detection results against ground-truth annotations	Work in Progress	Josiah
3.2.1.28	Memory efficiency	Disk usage remains below 85% while all data newer than the defined retention period is preserved without corruption	Run continuous data collection until storage limits are approached and verify automated deletion behavior through file timestamps and disk usage logs	Planned	Josiah
3.2.1.29	Network handling	The system automatically reconnects within 5 seconds of a network interruption without crashing or losing more than one message.	Intentionally disrupt TCP connections during operation and monitor reconnection time, stability, and message continuity.	Planned	Josiah
3.2.1.14	Fault tolerance	The system enters a safe degraded state during faults and fully recovers within 30 seconds without data corruption.	Inject simulated low-visibility inputs, disconnect subsystems, and perform abrupt power cycles while observing recovery behavior and data integrity	Planned	Josiah
3.2.1.30	Full integration	The fully integrated system runs continuously for at least one hour with stable connections and resource usage below defined limits	Operate the LiDAR and camera processing systems simultaneously in a live environment while monitoring performance metrics and system logs	Planned	Josiah
3.2.1.17	YOLOv8 Bee Detection Accuracy	Model achieves ≥ 90% mAP@0.5 on validation set of bee images	Run YOLOv8 on labeled validation images, calculate mAP@0.5 using Ultralytics metrics	Work in Progress	Jason
3.2.1.18	False Positive Rate on Backgrounds	≤ 1 false positive per 50 field/background (no bee) images	Evaluate model on negative images, log detection count; pass if rate is below threshold	Work in Progress	Jason
3.2.1.19	TCP Output Transmission	≥ 95% of detection outputs are successfully transmitted to Raspberry Pi with correct format	Simulate field workflow, send output via TCP, check logs on RPi	Planned	Jason
3.2.1.20	Multi-Bee Detection Robustness	In images with ≥ 2 bees, all bees are detected, boxes overlap with ground truth by ≥ 0.5 IoU	Test model on annotated multi-bee images, manually or script verify IOU for all boxes	Planned	Jason
3.2.1.21	End-to-End Data Flow	Frontend displays detection results (bounding boxes + species) within 2 seconds of image upload	Demo: upload image via web app, timed observation of bounding box/species overlay	Planned	Jason
3.2.1.22	New Species Expansion	Model retrain accepts new annotated classes, adds them to predictions, and maintains at least 85% accuracy for bee class	Add new species, retrain, check detection/classification results for all classes	Planned	Jason
3.2.1.23	Web Page Responsive Validation	Web page layout renders correctly and is usable on mobile, tablet, and desktop devices	Access the web page on various devices and screen sizes; verify layout, controls, and content for each device	Work in Progress	Jason

Thank you for listening!